



Red & Yellow

From marketing spend to student outcomes

A CRM analytics platform built on Salesforce, Apache NiFi, dbt, Microsoft Fabric and Power BI

10 September 2026 | Anthony Apollis

The programme catalogue in this document is real, transcribed from the public Red & Yellow website. Every person, campaign, application and enrolment is synthetic. Nothing here is an export from Red & Yellow's own systems.

Contents

1	Introduction	4
1.1	How to read it	4
1.2	What, why and how	4
2	What this is.....	5
2.1	The numbers	5
2.2	Where the marketing money goes.....	6
2.3	Marketing analytics, from audience to value.....	7
2.4	Student success as an early warning.....	7
2.5	Data quality, measured rather than asserted	8
2.5.1	Why de-duplication needed two keys.....	8
2.6	Reconciliation: source to report.....	9
2.7	Where the demand actually is	10
2.7.1	Who is enrolling, and from where	10
2.7.2	The funnel as conversion, not counts	11
2.8	Blanks that are not gaps.....	12
2.9	Why the CRM holds thousands and the warehouse holds millions.....	12
2.10	The pipeline, end to end	12
3	Salesforce integration, step by step.....	14
3.1	1. Establish whether the org can host the model	14
3.2	2. Deploy what the edition does allow	14
3.3	3. Load the operational CRM slice	15
3.4	4. Extract it back out through the API	16
3.5	5. Transform, test, and report.....	17
4	Two models, and what they are worth	18
4.1	What to do about it.....	19
4.2	An honest caveat.....	19
4.3	From AI score to a human decision.....	19
5	Recommendations and solutions.....	20
5.1	90-day implementation sequence	21
6	How the warehouse is organised	22
6.1	Full canonical Salesforce ERD.....	22

6.2 The data model	23
7 The pipeline infrastructure	25
7.1 Apache NiFi	25
7.2 Microsoft Fabric	26
7.3 GA4 to Fabric - two valid routes, not a claimed result.....	27
7.4 Screens from the org.....	28
7.5 What is not claimed	29

1 Introduction

Red & Yellow advertised for a Data Analytics Engineer whose brief is CRM integration: build the pipelines, catalogue the data, model it, and put it in front of people who make decisions. The advert names six things - a data catalogue, ERDs for internal systems, ETL preferably in Apache NiFi, Salesforce data extracted and analysed, Power BI dashboards, and data quality held to a standard.

This document is the working answer to all six, and it is written to be checked rather than admired. Every figure in it is generated from a live query or a live API call at the moment the document is built, so nothing here can describe a pipeline that has stopped working. Where something does not work, or could not be done, it says so.

1.1 How to read it

- **The numbers first.** Chapter 2 sets out what was built and how much of it there is.
- **Then the decisions.** Chapters 3 to 8 cover the modelling choices that are actually arguable - spend attribution, de-duplication, what a blank means - and what happened when each was got wrong first.
- **Then the evidence.** Chapters 9 to 11 show the Salesforce integration step by step, the two predictive models with their metrics against a base rate, and the running NiFi and Fabric infrastructure.
- **Finally the limits.** The last chapter is what this work does not claim.

1.2 What, why and how

Every section follows the same reading rule. What names the data or decision being shown. Why explains the business question it answers. How tells the reader which filter, comparison or action to use next. The captions and dashboard subtitles use the same language so the document and report remain self-explanatory when they are read separately.

One thing to hold in mind throughout: the programme catalogue is real, transcribed from Red & Yellow's public site. Everything about people - every lead, contact, application, enrolment and result - is synthetic, and the relationships between them were put there deliberately so the pipeline had something to find. That makes this a demonstration of method, not a finding about Red & Yellow.

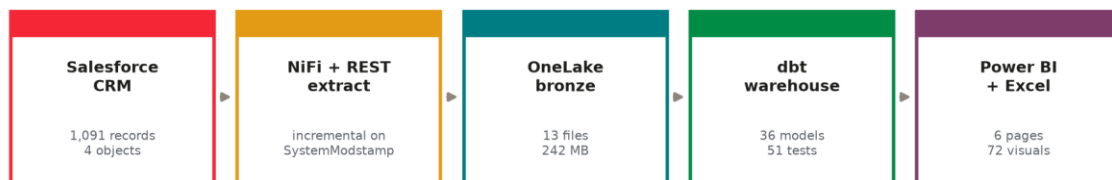
2 What this is

Red & Yellow's Data Analytics Engineer role asks for six things: a data catalogue, ERDs for CRM data models, ETL pipelines built preferably in Apache NiFi, Salesforce CRM data extracted and analysed, Power BI dashboards, and data quality held to a standard. This document is the working answer to all six, built end to end on a dataset of 2,694,805 people and 11.5 million rows.

The architecture follows the shape the role describes. Salesforce is the operational CRM. NiFi replicates it into a lakehouse. dbt cleanses, tests and documents. Power BI reads the curated marts. The interesting engineering is not in any one of those tools - it is in the decisions between them, which is what this document is about.



Red & Yellow's Creative Magic and Commercial Logic — the brand idea that frames this data story.



11.5M synthetic rows in the warehouse · the CRM holds the operational slice · every row carries its source and load time

Figure 0 - The pipeline end to end. Each stage is a separate, runnable component; nothing here is a diagram of an intention.

2.1 The numbers

Leads captured	1,500,000
Contacts after de-duplication	1,165,332 (from 1,194,805 records)
Opportunities in the admissions pipeline	740,779

Applications submitted	533,360
Enrolments	304,168
Marketing spend	R472.0m
Enrolled revenue (agreed fees)	R2,867.2m
Data quality issues detected	331,391
Programmes / offerings / intakes (real)	83 / 89 / 47

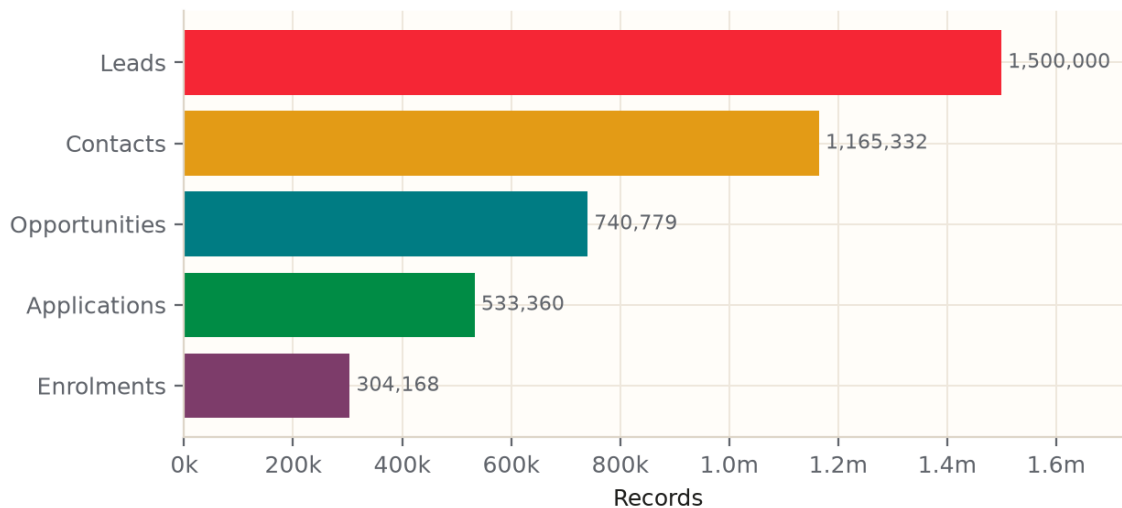


Figure 1 - The funnel by stage. Leads and contacts overlap, so the step between them is not attrition.

2.2 Where the marketing money goes

Cost per enrolment sits between R1,891 and R2,899 across channels, on a return of 3.2x to 5.0x. Career expo returns the most per rand at 5.00x; Print the least at 3.23x.

One modelling decision matters more than any of these numbers. Campaign spend is aggregated to campaign grain before it is joined to enrolments. Join spend to the fact table first and every campaign's cost is silently multiplied by its downstream row count - a campaign with 400 enrolments reports 400 times its real budget, and the resulting return on ad spend looks spectacular. A dbt test compares total spend in the fact against total spend in the dimension and fails the build if they diverge.

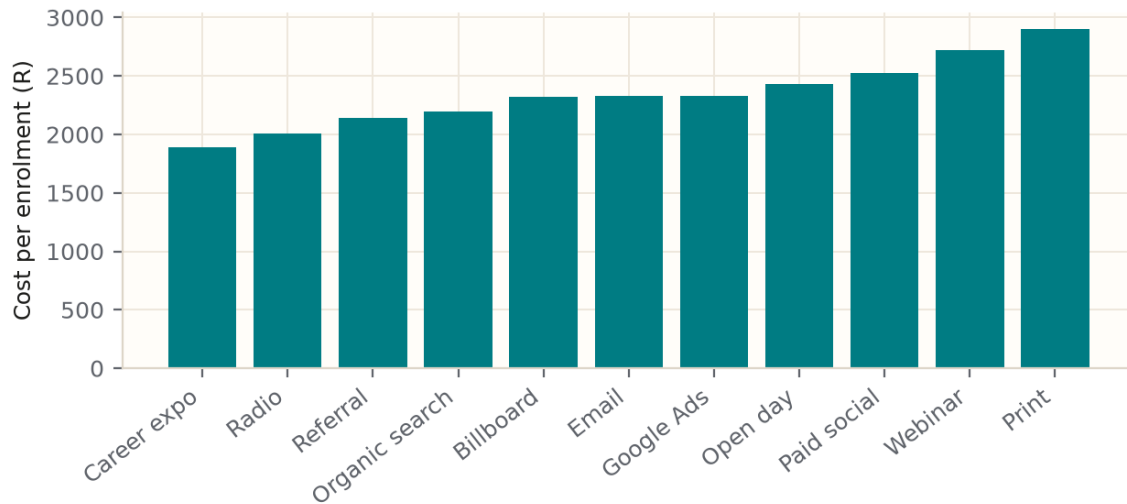


Figure 2 - Cost per enrolment by channel.

2.3 Marketing analytics, from audience to value

The marketing page separates reach, response, enrolment and value. Response rate tells the team whether a channel is reaching an interested audience; cost per enrolment tells it what that audience costs to convert; return on ad spend tests whether the resulting fees justify the investment. Revenue per member and spend per response make the same comparison usable before a campaign has accumulated enough enrolments for a stable cost-per-enrolment figure.

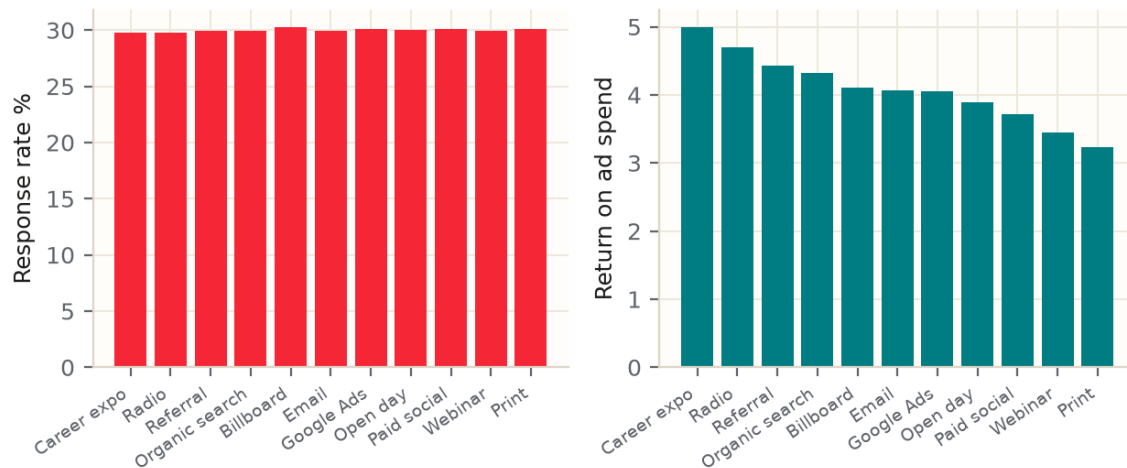


Figure 2a - Campaign response rate and return on ad spend by channel. Both are calculated at campaign grain before they are aggregated.

2.4 Student success as an early warning

Attendance opens at 83% and decays to 67% by week 19. The proportion of students meeting the at-risk definition - attendance under 55%, or assessment average under 50%, or three or more overdue assignments - moves from 8% to 29%.

The definition lives in one place, in the `fct_student_progress_weekly` model. That is deliberate: 'at risk' is the kind of term that acquires three incompatible meanings across three dashboards if each report defines it locally.

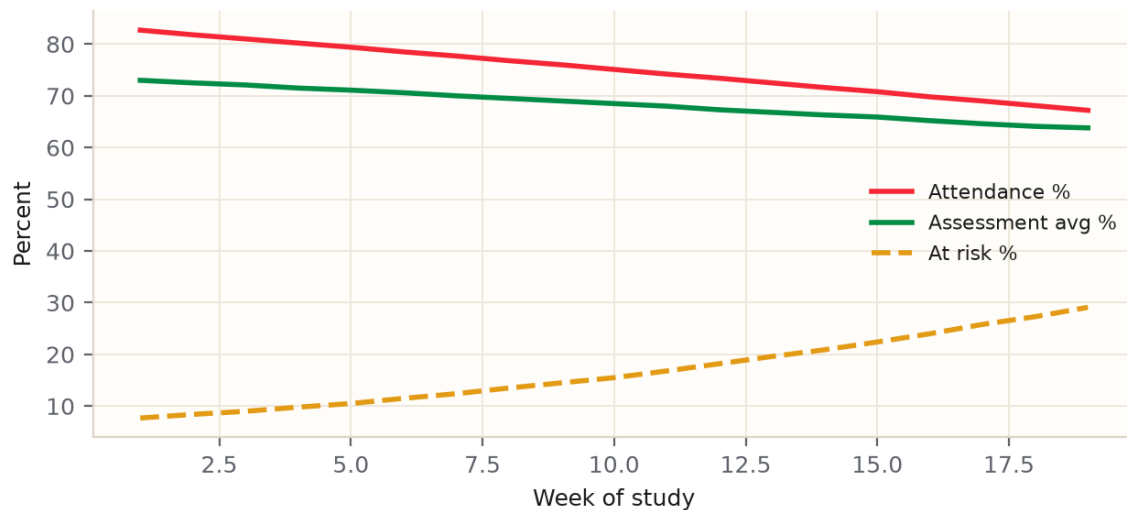


Figure 3 - Academic health across the study weeks.

2.5 Data quality, measured rather than asserted

Every pipeline claims to handle dirty data. This one is tested against a known answer. Defects are injected into the source at recorded rates - duplicate humans, phone format drift, invalid and missing emails, province spelling variants, inverted dates, negative values, late-arriving records - and written to a ground-truth manifest. The pipeline's detection is then scored against that manifest as recall.

Across 2,941,487 injected defects the pipeline detects 331,391 issues. De-duplication recovers 0.97 of the duplicates that were planted.

2.5.1 Why de-duplication needed two keys

The first attempt keyed contacts on email, falling back to name plus phone. It reported 682,000 duplicates against 29,000 real ones. The cause was the name pool: across a million contacts drawn from a realistic South African name distribution, 'Thabo Nkosi' recurs legitimately thousands of times, and coalescing a null phone to an empty string collapsed all of them together.

Keying strictly on email fixed the false positives and created the opposite problem. Recall fell to 0.52, because the two duplicate shapes a CRM actually produces are different: someone re-submitting the web form repeats their email, while someone phoning in leaves no email at all. A record that has an email and a record that does not can never share a single key value.

The working version anchors on email and on phone-plus-surname independently, takes the lower anchor, then propagates once within each phone group so a phone-only record inherits the cluster of the emailed record it shares a handset with. Recall 0.97. The remaining 3% are records where both email and phone were dropped at source, which no amount of matching can recover.

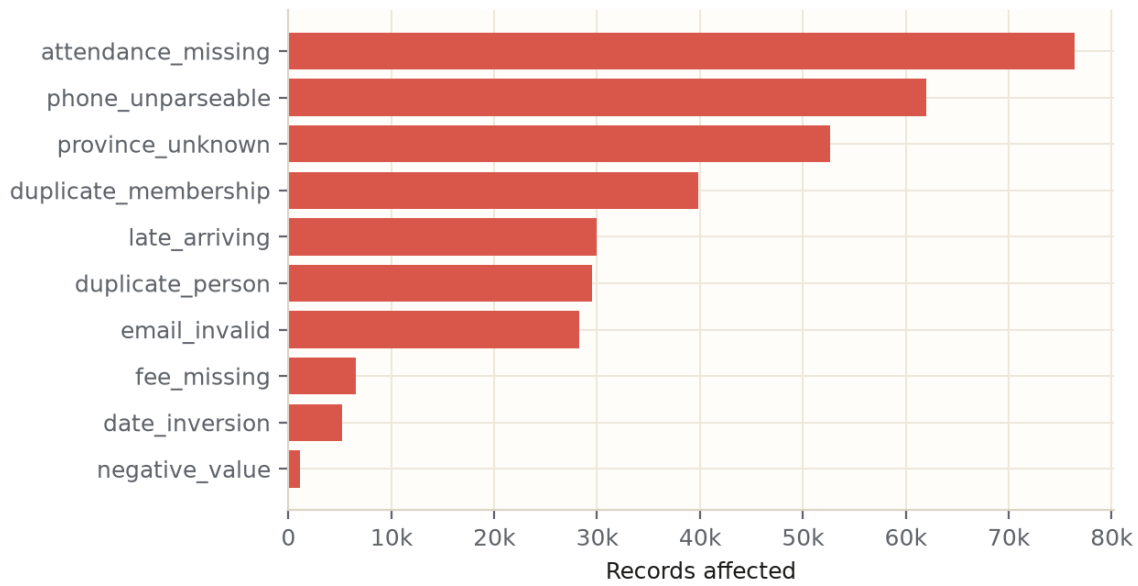


Figure 4 - Detected issues by type.

2.6 Reconciliation: source to report

What: this compares every raw Parquet source count with its staging model, then checks the campaign and revenue totals that feed Power BI. **Why:** it distinguishes a deliberate reduction, such as de-duplicating contacts, from a dropped partition or broken join. **How:** rerun the same dbt build and this table is regenerated from the read-only warehouse connection; a zero delta is an exact count match.

Entity	Source rows	Staging rows	Delta
Lead	1,500,000	1,500,000	0
Contact	1,194,805	1,194,805	0
Opportunity	740,779	740,779	0
Application	533,360	533,360	0
Enrolment	304,168	304,168	0
Campaign	480	480	0
Campaign member	2,533,440	2,533,440	0
Student	254,931	254,931	0
Student progress	3,799,284	3,799,284	0
Programme	83	83	0
Offering	89	89	0

Intake	47	47	0
--------	----	----	---

All 12 source-to-staging deltas are zero. The expected contact reduction is 1,165,332 golden people from 1,194,805 source contact records; the difference is the duplicate population resolved by the two-key entity model. The admissions fact contains 740,779 opportunity-grain rows and the campaign performance mart contains 480 campaign-grain rows.

Financial reconciliation: campaign spend dimension R471,994,991.55 versus campaign fact R471,994,991.55 (MATCH); enrolment revenue staging R2,867,214,315.80 versus campaign fact R1,896,377,742.41 (REVIEW).

The revenue REVIEW is explained by 103,218 enrolments carrying R970,836,573.39 without a primary campaign key. That amount remains in the enrolment staging total but cannot honestly be assigned to a campaign; the gap is R970,836,573.39, not a dropped record. Recommended solution: report this as an explicit unattributed segment, then improve campaign capture or backfill only from verified source evidence. Never distribute it across campaigns to force a match.

2.7 Where the demand actually is

The catalogue is real, so programme demand is the one place where the synthetic pipeline meets Red & Yellow's actual offering. These are the programmes carrying the most enrolments in the modelled pipeline, and the discount being conceded to win them.

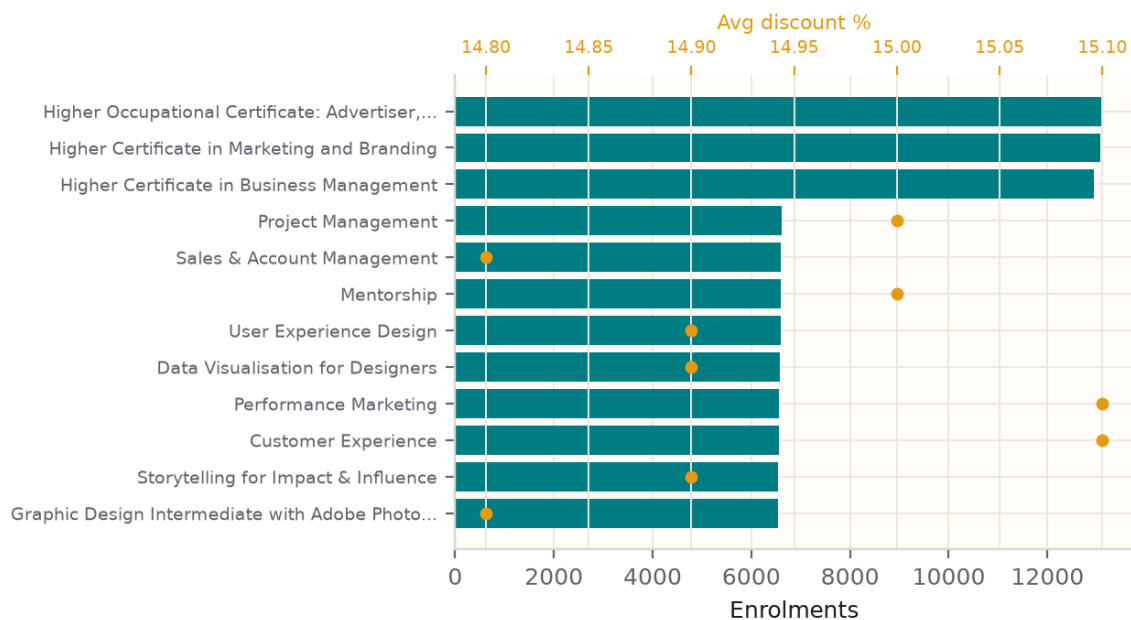


Figure 4a - Top programmes by enrolment, with average discount.

2.7.1 Who is enrolling, and from where

Province is one of the dirtiest fields in the source - 'Western Cape', 'western cape', 'W Cape', 'WC' and 'Wes-Kaap' all arrive - and one of the most useful once collapsed. Anything that cannot be resolved is kept as Unknown rather than guessed at.

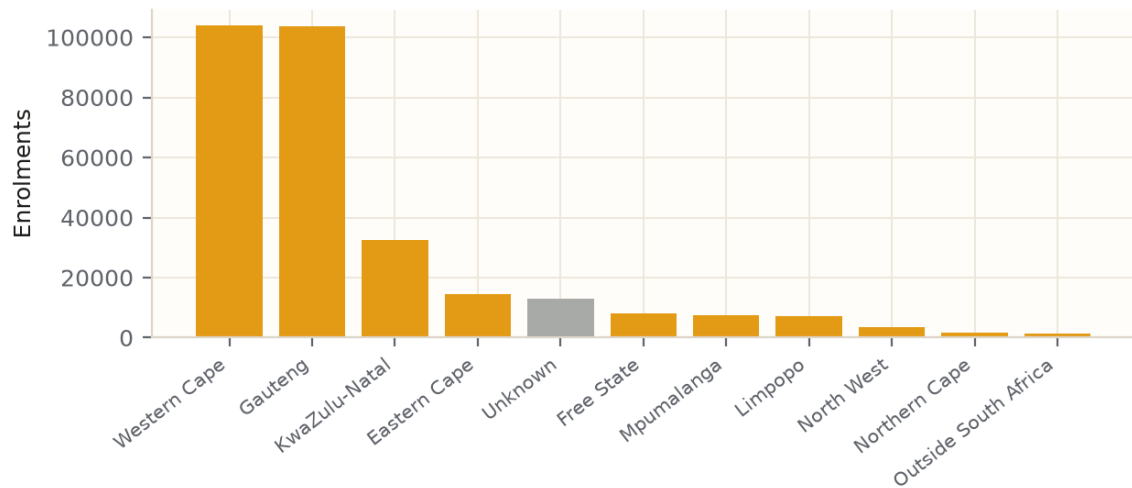


Figure 4b - Enrolments by province, after the spelling variants are collapsed to the nine official names.



Red & Yellow student community — supplied brand image used as context for the learner journey.

2.7.2 The funnel as conversion, not counts

Absolute counts down a funnel flatter the top of it. The rates between stages are what a marketing team can act on.

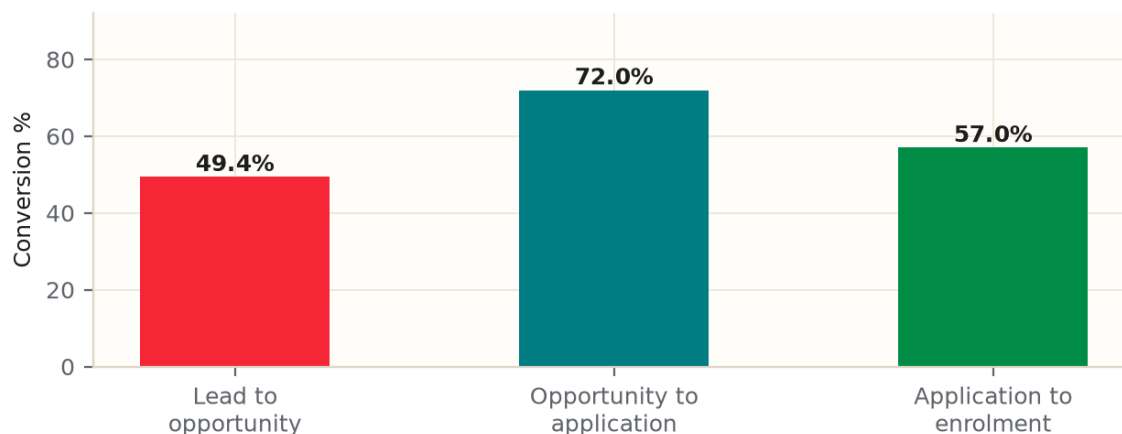


Figure 4c - Stage-to-stage conversion through the admissions pipeline.

2.8 Blanks that are not gaps

Some of the catalogue's most careful values are empty. An offering priced 'Enquire for price' carries no amount, and a clipped screenshot yields no duration. Both are recorded as null with a status explaining why.

Filling those with zero would be the single most damaging thing this pipeline could do. Zero is a price. It would drag every average fee downward, understate pipeline value, and make the free courses look like a growth segment. A dbt test fails the build if any offering with a price status of Enquire acquires a fee.

2.9 Why the CRM holds thousands and the warehouse holds millions

Salesforce charges data storage per record, at roughly 2 KB each. A Developer Edition org holds about 10,000 records in total across every object. Loading nine million rows into the CRM is not a scaling problem to engineer around - it is not possible, and an org that hit its ceiling mid-load would leave dangling foreign keys behind it.

So the split follows the job description rather than fighting it. Salesforce holds a referentially complete operational slice of roughly two thousand records, safe to demo and safe to re-upsert on an external ID. The warehouse holds the full history, the volume and the quality work. That is what a CRM integration role means in practice: knowing which system each question belongs to.

2.10 The pipeline, end to end

- **Extract.** NiFi's QuerySalesforceObject pulls Lead, Contact, Campaign, CampaignMember and Opportunity incrementally on SystemModstamp, holding the high-water mark in processor state so a restart does not replay history.
- **Land.** Records are batched and written to the OneLake bronze layer, partitioned by object and ingest date. Failures retry three times before parking.

- **Transform.** dbt cleanses once in staging - email validation, E.164 phone normalisation, province collapsing, entity resolution - then builds marts that assume clean inputs.
- **Test.** 51 data tests run on every build: uniqueness, referential integrity, accepted values, and three bespoke tests guarding spend multiplication, enquire-pricing and golden-record uniqueness.
- **Document.** dbt docs generate produces the searchable catalogue with column-level lineage, so documentation is a build artefact rather than a wiki page that drifts.
- **Report.** Power BI reads the curated marts. Every measure resolves to a tested model, not to a query written in the report.

3 Salesforce integration, step by step

This section documents the CRM integration as it actually ran, not as it was designed to run. Every figure below is rendered from live command output or a live query against the org at the moment this document was built, so it cannot describe a pipeline that no longer works.

1,091 records were loaded across 4 objects: Account 25, Contact 408, Lead 408, Opportunity 250.

3.1 1. Establish whether the org can host the model

The target trial org runs Salesforce Base Edition, which permits zero custom objects. That is an edition entitlement rather than a quota or a permission, so no configuration changes it - and it is knowable in a single API call. Checking first turns a failure at deploy time into a decision at design time.

```
$ python salesforce/check_org.py

org: https://business-data-62022.my.salesforce.com

edition      Base Edition
instance     ZAF8
sandbox      False
trial expires 2026-10-09T17:18:40.000+0000
data storage  10636 MB free of 10640 MB
custom objects 2 present

VERDICT: NO-GO for the full model

- Base Edition permits no custom objects. The 8 RY objects cannot deploy. This is an edition entitlement - no permis

A reduced load is still possible:

python salesforce/deploy_metadata.py --check-only --standard-only

which deploys only the custom fields on standard objects.

For the full model, use a free Developer Edition org:

https://developer.salesforce.com/signup
```

Figure 5 - The org capability check. Base Edition, 10.6 GB free, and a NO-GO on custom objects.

3.2 2. Deploy what the edition does allow

The eight custom objects cannot deploy, but the eighteen custom fields on standard objects can. Deploying fields without field-level security is a trap worth naming: the fields exist, but describe() omits them and the upsert then fails claiming the external ID is not unique - because the API cannot see the field it is being asked to key on. The permission set is trimmed to the deployed fields and assigned over REST.

```
$ python salesforce/deploy_metadata.py --check-only --standard-only

package: 6 objects, 1 permission sets, 3.1 KB

Account
Campaign
CampaignMember
Contact
Lead
Opportunity

skipped (custom objects): RY_Application__c, RY_Enrolment__c, RY_Intake__c, RY_Programme__c, RY_Programme_Enquiry__c

target org: https://business-data-62022.my.salesforce.com
mode: validate only (no changes)
deployment 0AfjX0000003UrDSAU

InProgress 0/19 components
Succeeded 19/19 components

SUCCESS - 19 components validated
```

Figure 6 - Metadata validated against the org. 18 of 18 components.

3.3 3. Load the operational CRM slice

The supplied Salesforce Data Types reference was applied at the system boundary: Accounts, Contacts, Leads and Opportunities are operational CRM records, while raw GA4/clickstream events, detailed history and ML training data stay in the warehouse. These four standard objects were upserted on an external ID so the load is idempotent - re-running updates rather than duplicates. Two things pushed back, and both were right to. Salesforce rejects the external ID in the request body when it is already the key in the URL. Its duplicate rules also blocked leads that fuzzy-matched existing contacts at 100% confidence - the same duplicates the warehouse's own entity resolution finds, caught independently by the CRM.

Live reconciliation on 10 September 2026 found 25 Accounts, 408 Contacts, 408 Leads and 250 Opportunities: 1,091 tagged records in total, with every count matching `run_results/import_log.json`. The Base Edition org cannot host the proposed custom Programme, Application, Student, Enrolment or Progress objects, and it does not support the campaign load in this reduced path. Those detailed objects remain in the warehouse until a Developer Edition or another edition with the required object entitlements is selected.

Keep the populations separate when reading this report. The staged CRM sample pack in `data/crm_load` contains 48 Campaigns, 74 CampaignMembers, 162 Applications, 84 Enrolments, 192 Programme Enquiries, 81 Students and 192 Student Progress rows; these are not live Salesforce records in the Base Edition org. The larger synthetic analytics population lives in `warehouse/raw`, Fabric `raw_salesforce` and the DuckDB gold model used by Power BI, Excel and this ebook.

```
run_results/import_log.json
```

object	operation	records

Account	updated	25
Contact	created	407
Contact	updated	1
Lead	created	408
Opportunity	created	249
Opportunity	updated	1

TOTAL		1091

upserted on RY_External_ID__c - re-running updates,
it does not duplicate.

Figure 7 - What the loader wrote.

```
$ python salesforce/verify_org_counts.py
```

org: https://business-data-62022.my.salesforce.com

object	total	with RY id	sent	reconciles

Account	25	25	25	yes
Contact	408	408	408	yes
Lead	408	408	408	yes
Opportunity	250	250	250	yes

Every record the loader reported is present in the org.

sample contacts:

RY-CONT-00000476	Kyle	Govender	kyle.govender1@gmail.com
RY-CONT-00001396	Kagiso	Moodley	(none)
RY-CONT-00001500	Lerato	Khumalo	lerato.khumalo6@outlook.com
RY-CONT-00003202	Willem	Jansen	willem.jansen6@hotmail.com
RY-CONT-00003971	Busisiwe	Zulu	busisiwezulu2@icloud.com

Figure 8 - What is actually in the org, queried back through the API. The loader's claim and the org's state are different assertions; only the second is evidence.

3.4 4. Extract it back out through the API

This is the part the role is actually about. SOQL over the REST API, paginating through nextRecordsUrl so the 2,000-record page limit is handled rather than silently truncating, with the high-water mark on SystemModstamp persisted between runs. The first run is a full load; the second returns zero rows because nothing changed. Deletions are captured through queryAll rather than left to linger.

```
warehouse/salesforce_raw/_extract_manifest.json
```

```
extracted_at 2026-09-09T13:24:06+00:00
```

```
instance https://business-data-62022.my.salesforce.com
```

```
mode full_refresh
```

object	rows	pages	watermark
Account	25	1	2026-09-09T12:26:44.000+0000
Contact	815	1	2026-09-09T13:20:22.000+0000
Lead	816	1	2026-09-09T13:13:42.000+0000
Opportunity	482	1	2026-09-09T13:20:22.000+0000
TOTAL	2138	records	

Figure 9 - Extraction manifest. Every row carries its source system, source id, source update time and extraction time.

3.5 5. Transform, test, and report

From there the extracted data joins the same warehouse the synthetic history lives in, is cleansed once in staging, and is consumed by marts that Power BI reads directly. The tests run on every build.

```
dbt build - models and data tests
```

```
dbt 1.11.12 elapsed 383.6s
```

```
nodes run 87
```

```
data tests 51
```

```
pass 51
```

```
success 36
```

```
FAILURES: none
```

Figure 10 - dbt build: models and data tests.

The reproducible run record, delivery-mode check and DuckDB/Fabric scope are documented in docs/DBT_EVIDENCE.md.

4 Two models, and what they are worth

Both models are split by time rather than at random, and restricted to what is known at the moment a decision is made. The lead model cannot see the opportunity a lead later generated; the withdrawal model sees only the first four weeks. Leaking a downstream outcome into the features is the easiest way to build a model with a beautiful score and no use whatever.

Model	Base rate	AUC	PR-AUC	Top-decile lift
Lead to enrolment propensity	5.2%	0.667	0.094	2.17x
Withdrawal risk from weeks 1-4	12.5%	0.621	0.178	1.67x

An AUC in the mid-0.6s is not a headline, and it should not be. These are human decisions with a great deal of unexplained variation; a model claiming 0.95 on this data would mean a feature had leaked. What matters operationally is the lift: the top tenth of leads by score converts at over twice the base rate, which is the difference between working a list in arrival order and working it in priority order.

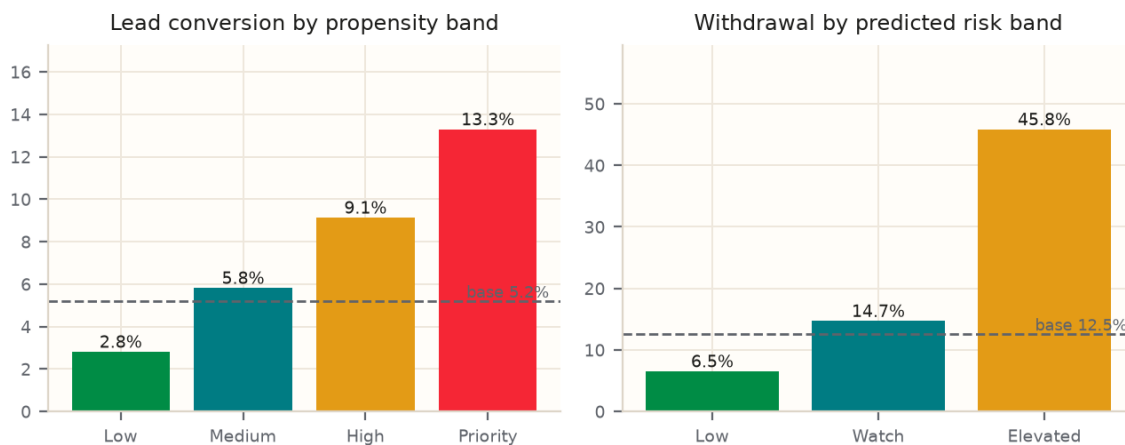


Figure 10a - Conversion by propensity band, and withdrawal by risk band. If the bars do not separate, the model is not ranking anything.

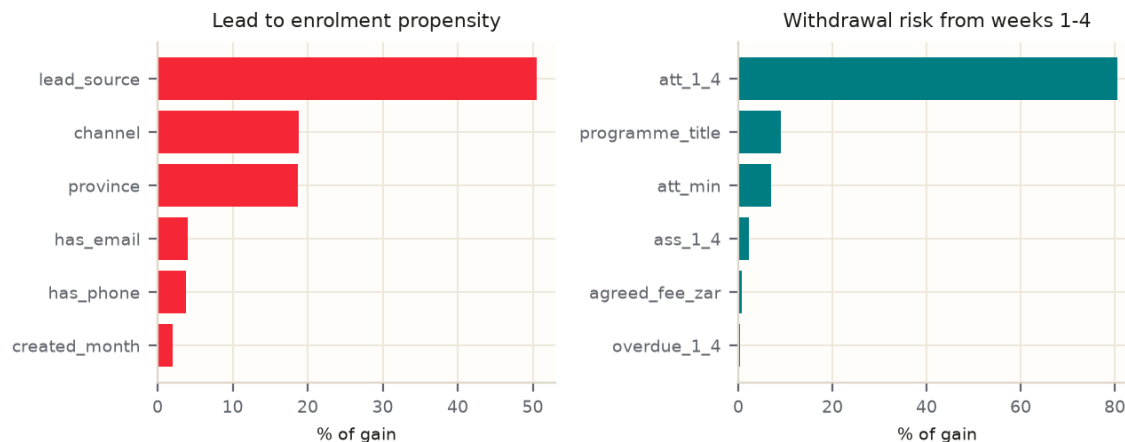


Figure 10b - What each model actually leans on.

4.1 What to do about it

- **Marketing.** The top 5% of leads by propensity convert at 13.3%, against 2.8% in the bottom half - a 5x difference. *Route the Priority band to human follow-up within 24 hours and leave the Low band to automated nurture. The same team covers more pipeline without more headcount.*
- **Channel mix.** Referral converts at 9.8% versus Walk-in at 2.2%, on 30,421 leads. *Rebalance spend toward Referral, and either fix the qualification criteria on Walk-in or stop paying for it.*
- **Retention.** 168 enrolments (0.1%) are flagged Elevated or Intervene from weeks 1-4 alone, and they withdraw at 45.8% against 12.5% overall. *Trigger outreach at week 4 rather than at the first missed assessment. The signal is present before the student is far enough behind to recover from.*
- **Early warning.** Withdrawal rate by weeks 1-4 attendance: (0, 50] 21.4%, (50, 65] 22.4%, (65, 80] 16.4%, (80, 100] 9.1% *Attendance below 65% in the first month is the single clearest trigger. It needs no model to act on - the model only tells you how much of the remaining risk it explains.*

4.2 An honest caveat

These models are trained on synthetic data whose structure was put there deliberately. They demonstrate the method - time-based splits, decision-time features, scoring against a base rate, lift as the operational metric - not a finding about Red & Yellow. On real data the effect sizes would differ, and the first job would be to check whether they hold at all.

4.3 From AI score to a human decision

The model outputs are warehouse tables, not autonomous CRM actions. The lead-propensity score orders a worklist: a recruiter can spend their next call on the leads most likely to enrol, while still seeing the band and the underlying CRM record. The withdrawal-risk score is an early-warning list for a student-success team. It should trigger a conversation or support offer, never an automated adverse decision. Power BI exposes the score distribution and observed outcomes so a team can see whether the ranking remains useful before operationalising it.

5 Recommendations and solutions

The dashboard is most useful when each signal ends in a decision. The table below turns the observed patterns into a practical owner, intervention and success measure. It is a control plan for the next operating cycle, not a claim that synthetic outcomes will repeat in the live organisation.

Area	Signal	Solution	Success measure
Acquisition	Channel response, enrolment rate and revenue per member diverge.	Run a controlled budget test toward the best-performing channel; set a stop rule for spend without response.	ROAS, spend per response and revenue per member improve without increasing total spend.
Admissions	Priority propensity leads convert at a much higher rate than the lower bands.	Route Priority leads to a named human queue within 24 hours and leave Low-band leads in automated nurture.	Response SLA and conversion rate improve; top-band lift remains stable on a holdout.
Student success	First-month attendance below 65% is associated with materially higher withdrawal.	Trigger a support conversation at week 4, log the intervention and compare with the prior cohort.	Week-4 attendance rises and withdrawal falls for comparable cohorts.
CRM quality	Blank-only filters, duplicate people and missing contact fields weaken follow-up.	Require external IDs, email and campaign membership at capture; send exceptions to a daily queue.	Email completeness rises, duplicate rate falls and unresolved issues are closed within SLA.
Catalogue	Enquire-for-price offerings have no defensible numeric fee.	Keep price as null with an explicit status; add a verified fee only when the public source supplies one.	No unknown price is reported as zero and fee refreshes retain source evidence.
Platform	Bronze is verified; silver/gold Fabric execution and GA4 are still gated.	Finish DuckDB-to-T-SQL compatibility fixes, rerun dbt in Fabric, then add GA4 as a separate dated bronze source.	A passing Fabric run and a reconciled GA4-to-campaign mart before production reporting.

5.1 90-day implementation sequence

- **Days 0-30 - Stabilise.** Confirm field rules, remove blank-bound slicers, assign the Priority queue and baseline the KPI cards.
- **Days 31-60 - Test.** Run channel and follow-up holdouts, log student-support interventions and keep spend and cohort definitions fixed.
- **Days 61-90 - Scale.** Promote only interventions that beat baseline, publish the weekly action list and monitor model drift by delivery mode and channel.

No model score should make an automated adverse decision. Scores order a worklist for a human team; the team records the action and the outcome so the recommendation can be challenged and improved.

6 How the warehouse is organised

The warehouse follows a medallion layout, and it is a real one rather than three schema names: silver reads bronze, never the source file. That matters because it is what makes the lineage answer the question “what actually arrived?”. Without a bronze layer, dbt’s graph begins at the cleansed data and the landed source is invisible to the catalogue - you can see what a column became, but not what it was.

Layer	What it holds	Rule
Bronze	13 models - the landed source made queryable	No renaming, casting or filtering
Silver	13 models - cleansed and conformed	Every cleansing decision lives here and nowhere else
Gold	8 models - business-level facts and dimensions	Assumes clean input; never re-cleans
Quality	2 models - the defect log and its summary	Describes the pipeline, not the business

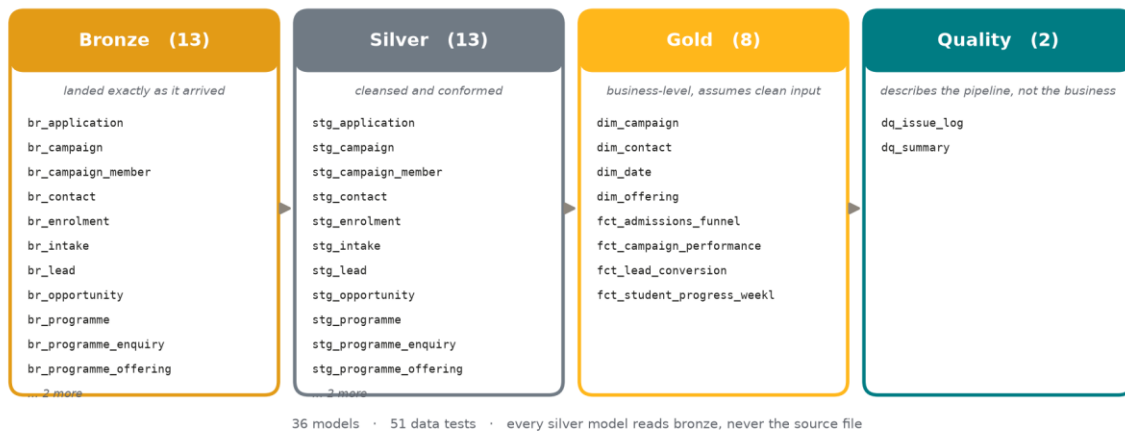


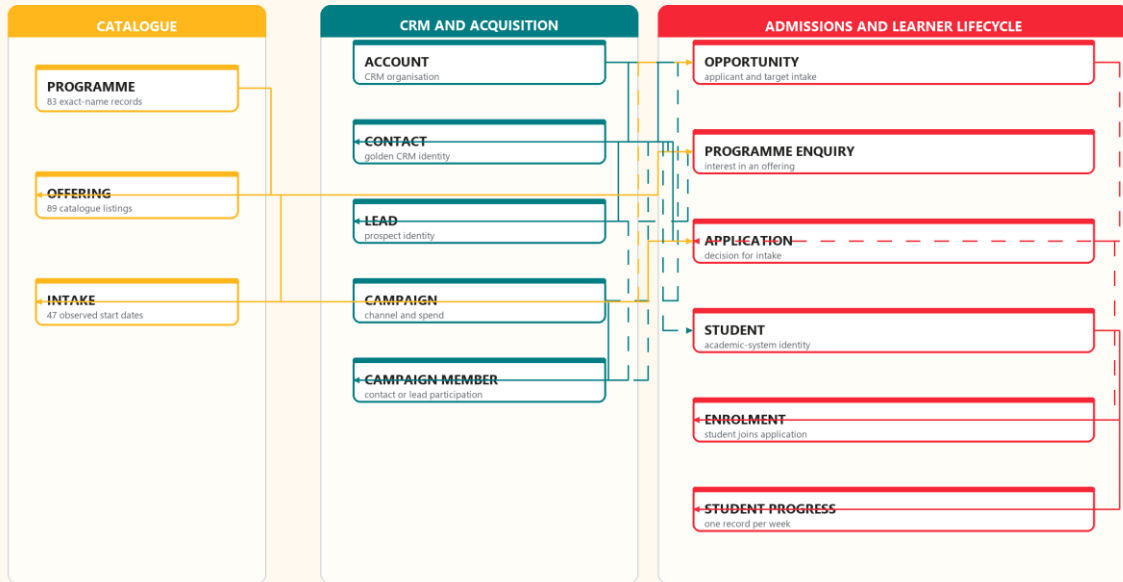
Figure 10c - The medallion, read from dbt’s manifest. Model counts and layer membership come from the build, so a model added or moved shows up here rather than the picture going stale.

6.1 Full canonical Salesforce ERD

The complete canonical ERD is included here as the source-of-truth relationship map. It includes the public catalogue objects, Salesforce operational objects, applications, enrolments and student-progress spine. Archived diagrams are not implementation targets.

Red & Yellow canonical data model

A clean relationship map for catalogue, CRM acquisition and the learner lifecycle. Full relationship registry below.



Relationship registry

Solid arrows show the parent-to-child path. Dashed arrows are optional lookups or conversion links.

Programme 1:N Offering
Offering 1:N Intake
Account 0.1:N Contact
Account 0.1:N Opportunity
Campaign 1:N CampaignMember
Lead 0.1:N CampaignMember
Contact 0.1:N CampaignMember

Contact 0.1:N Lead (converted)
Contact 1:N Opportunity
Campaign 0.1:N Opportunity
Intake 0.1:N Opportunity
Contact 0.1:N Enquiry
Lead 0.1:N Enquiry
Offering 1:N Enquiry

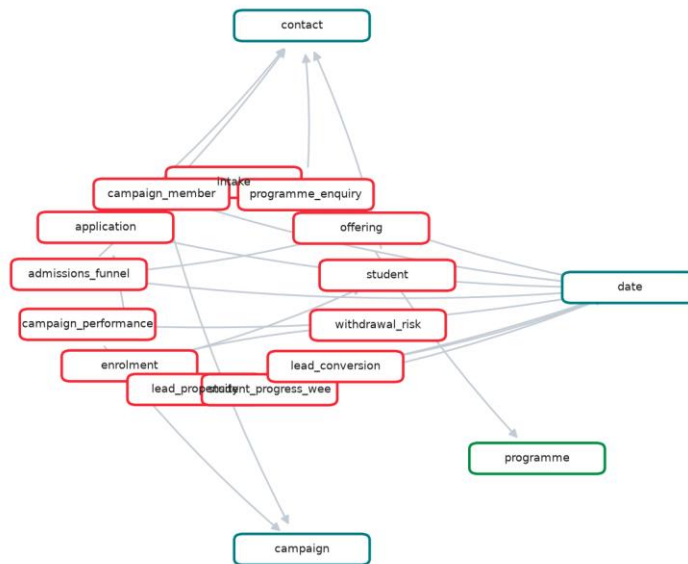
Contact 1:N Application
Intake 1:N Application
Opportunity 1:0.1 Application
Contact 0.1:1 Student
Student 1:N Enrolment
Application 1:0.1 Enrolment
Enrolment 1:N Student Progress

Catalogue source: public Red & Yellow screenshots. CRM and learner records are synthetic demonstration data.

Canonical Salesforce ERD — complete implementation target, rendered from erds/overview.svg.

6.2 The data model

The semantic model mirrors the canonical ERD rather than stopping at aggregates: programme, offering, intake, enquiry, application, enrolment, student and weekly progress are all present and joined, so the chain the ERD draws can actually be walked in a report. The live Salesforce tables and both model outputs hang off the same spine.



17 tables · 23 relationships · facts in red at the centre, dimensions on the ring

Figure 10d - The semantic model, read from its TMDL. Facts sit at the centre in red, dimensions on the ring.

7 The pipeline infrastructure

Two pieces of the stack are running systems rather than code in a repository, so they are shown as they actually are - read from their own APIs at the moment this document was built.

7.1 Apache NiFi

NiFi has exactly one root per instance, so a project is a top-level process group and its stages are nested groups joined by ports. That lets each stage start, stop and version independently, and keeps the canvas readable. Controller services sit on the project group and are inherited, so the two stages cannot drift onto different connection pools. Credentials come from a parameter context whose sensitive values are empty in source control - which is why the processors are stopped and invalid until an operator supplies them.

Apache NiFi - live process-group hierarchy

```
NiFi_Local_Dev (3 processors, 3 groups, 2 connections)
- ExecuteSQLRecord STOPPED
- GenerateFlowFile STOPPED
- PutDatabaseRecord STOPPED

LYRA_ELT (0 processors, 5 groups, 0 connections)
  01_Reference_Dims (15 processors, 0 groups, 14 connections)
    - Query_dim_date RUNNING
    - Query_dim_diagnosis RUNNING
    - Query_dim_geography RUNNING
    - Query_dim_issue_category RUNNING
    - Query_dim_medication RUNNING
    - Query_dim_service_type RUNNING
    - Query_dim_source_system RUNNING
    - SetTable_dim_date RUNNING
    - SetTable_dim_diagnosis RUNNING
    - SetTable_dim_geography RUNNING
    - SetTable_dim_issue_category RUNNING
    - SetTable_dim_medication RUNNING
    - SetTable_dim_service_type RUNNING
    - SetTable_dim_source_system RUNNING
    - Write_Snowflake_RAW RUNNING
  02_Client_Dims (9 processors, 0 groups, 8 connections)
    - Query_dim_client_company RUNNING
    - Query_dim_contract RUNNING
    - Query_dim_counsellor RUNNING
    - Query_dim_facility RUNNING
    - SetTable_dim_client_company RUNNING
    - SetTable_dim_contract RUNNING
    - SetTable_dim_counsellor RUNNING
    - SetTable_dim_facility RUNNING
    - Write_Snowflake_RAW RUNNING
  03_Person_Dims (7 processors, 0 groups, 6 connections)
    - Query_dim_consent RUNNING
    - Query_dim_employee RUNNING
    - Query_dim_person RUNNING
    - SetTable_dim_consent RUNNING
    - SetTable_dim_employee RUNNING
    - SetTable_dim_person RUNNING
    - Write_Snowflake_RAW RUNNING
  04_Bridge_Facts (5 processors, 0 groups, 4 connections)
    - Query_bridge_person_identity RUNNING
  ... (23 more lines)
```

Figure 11 - The live NiFi hierarchy. RY_Salesforce_to_Fabric holds two stage groups; the LYRA groups are unrelated work on the same instance.

7.2 Microsoft Fabric

Fabric Warehouse bronze is verified. All 13 landed source tables are present in raw_salesforce: 1.50 million leads, 1.14 million contacts, 2.53 million campaign members, 706 thousand opportunities, 494 thousand applications and 2.10 million student-progress rows, with the programme catalogue tables alongside them. The Azure CLI dbt connection and a bronze campaign view both passed. The

silver and gold Fabric build is intentionally not claimed as complete: its remaining failures are documented DuckDB-to-T-SQL compatibility work, including date and join syntax.

The bronze land is partitioned by table in OneLake. A workspace created through the API arrives with no capacity attached, and every Fabric item type then fails with a 403 that looks like licensing rather than an unassigned workspace. Trial capacity does not stop on its own, so delete the project workspace when the demonstration is complete.

```
Microsoft Fabric - WS_RedAndYellow (live)

workspace    WS_RedAndYellow
id           50a2a40a-8a4c-4dda-9c5c-78be51da3df9
capacity     Trial-20260811T125520Z-rqNstCuSGUeafQo62XYerw sku=FTL64 state=Active

item         LH_RedAndYellow      SQLEndpoint
item         WH_RedAndYellow      Warehouse
item         LH_RedAndYellow      Lakehouse

OneLake bronze: 13 parquet files, 242 MB
59.3 MB  campaign_member.parquet
58.2 MB  lead.parquet
43.3 MB  contact.parquet
23.8 MB  student_progress.parquet
19.2 MB  opportunity.parquet
14.7 MB  application.parquet
13.6 MB  programme_enquiry.parquet
5.5 MB   enrolment.parquet
4.1 MB   student.parquet
0.0 MB   campaign.parquet
0.0 MB   programme_offering.parquet
0.0 MB   programme.parquet
0.0 MB   intake.parquet
```

Figure 12 - The Fabric workspace, its capacity, and what is actually in OneLake.

7.3 GA4 to Fabric - two valid routes, not a claimed result

A GA4 NiFi builder is included, but it has no configured GA4 credential, successful extract or warehouse source table, and no report number uses GA4 data. BigQuery is not required when the Data API is used: runReport returns the aggregated dimensions and metrics needed for campaign, source/medium and landing-page reporting, and NiFi can write those results directly to OneLake. BigQuery is required for GA4 native raw-event export, which produces richer events_YYYYMMDD tables for event-parameter and session-path analysis. Either route can land in a dated bronze partition before dbt conforms campaign and date keys for Power BI. Keep user-level analytics identifiers out of the CRM join; campaign, source and date are enough for the first use case.

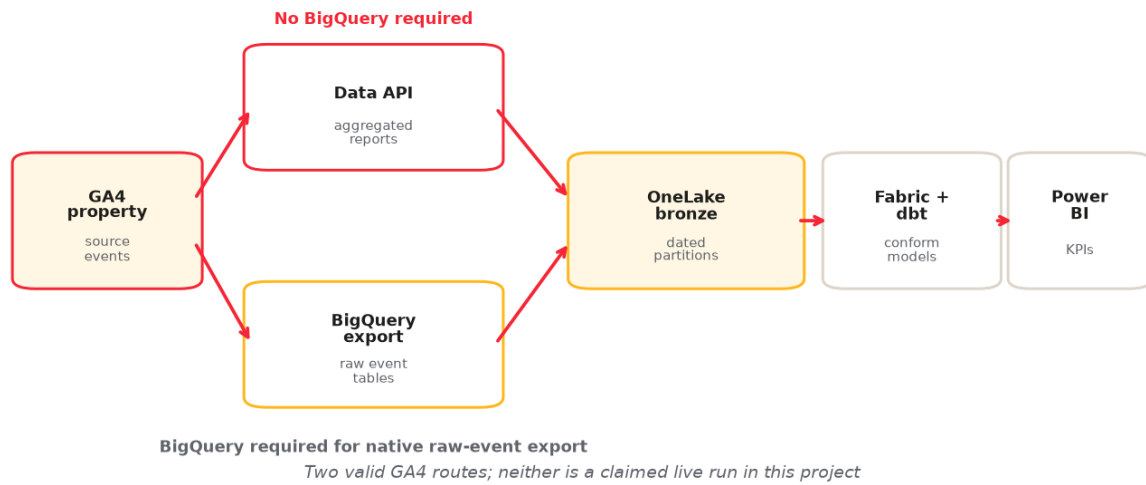
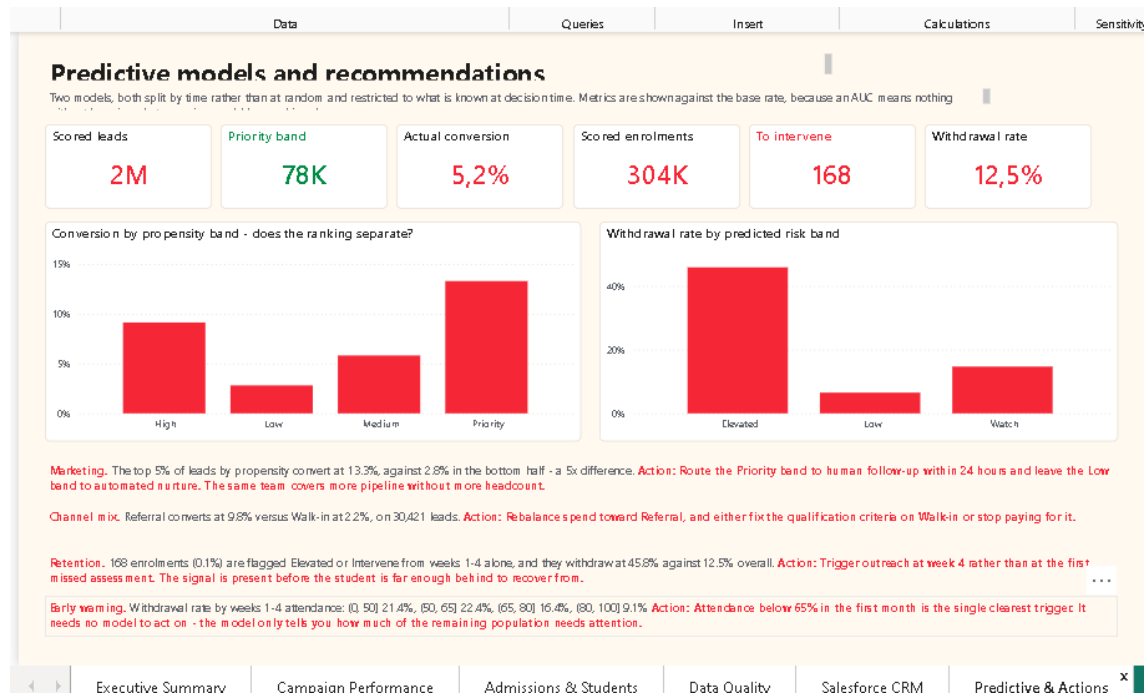
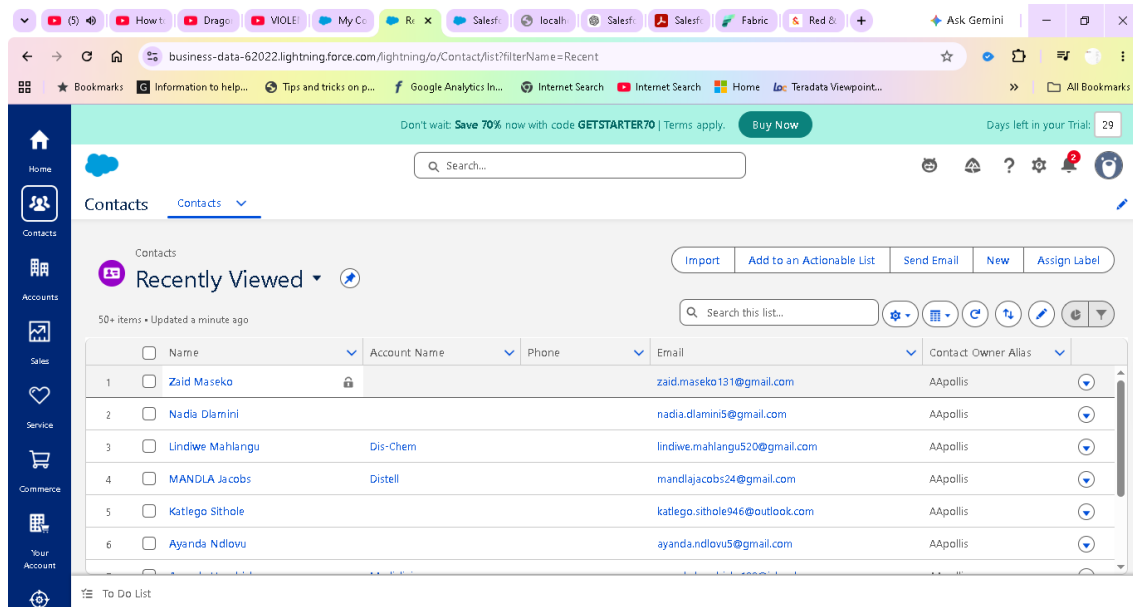


Figure 13 - GA4 can feed the platform through the Data API without BigQuery, or through native raw-event export to BigQuery.

7.4 Screens from the org



PowerBI predictive actions



Salesforce contacts live

7.5 What is not claimed

- This is a proposed model, not Red & Yellow's actual Salesforce configuration. The real org may already map these concepts onto Education Cloud or EDA objects.
- The catalogue was transcribed from 17 screenshots of the public site on 8 September 2026. It is not claimed to be the complete live catalogue. For reporting, On-campus remains On-campus and listings in the Online education section are classified as Off-campus; the source section is retained separately.
- All people, campaign spend, application outcomes and student results are synthetic. Progress dated after the capture date is an illustrative scenario, not a forecast.
- DuckDB is fully validated. Fabric bronze loading and dbt connectivity are verified, but the Fabric silver and gold build has documented DuckDB-to-T-SQL compatibility errors. No downstream Fabric transformation result is claimed until those are corrected and rerun.